

## Implementação do KNN usando OpenMP

O hardware utilizado foi um Intel Core i5 - 1345U, executado em um notebook, também é necessário informar que o código foi executado dentro de um *docker container*, rodando pelo WSL do Windows.

O **problema**, dado um set de pontos bi-dimensionais  $S$ , divididos em  $n$  grupos, um inteiro  $k$  e um ponto bi-dimensional  $P$  para ser classificado, o algoritmo KNN pode ser usado para resolver a classificação de  $P$ . O algoritmo funciona comparando as distancias entre cada ponto de  $s \in S$  para o ponto  $P$ , selecionando os  $k$  pontos com a menor distância e contando a frequência em que cada grupo, entre esses  $k$  pontos mais próximos, aparece com maior frequência.

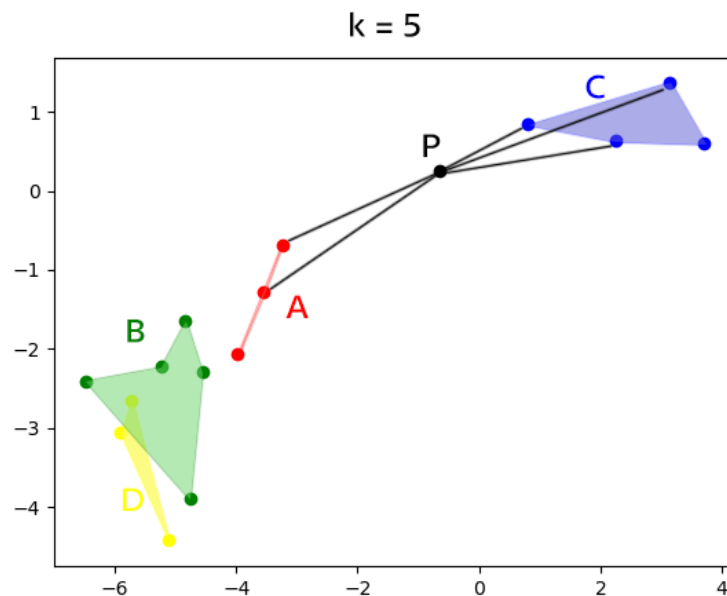


Figura 1: Exemplo simples da classificação com KNN

O algoritmo KNN fornecido, como mostra o Trecho de Código 1, a instrução *for* utilizada na linha 11, ordena o *array*  $k$ , que possui o mesmo tamanho de  $k$ .

```

1 // ...
2
3 for (i = 0; i < n_groups; i++) {
4   Group g = groups[i];
5
6   for (j = 0; j < g.length; j++) {
7     float d = euclidean_distance_no_sqrt(to_evaluate, g.points[j]);
8
9     for (x = 0; x < k; x++) {
10      if (d < distances[x] || distances[x] == -1) {
11        for (y = k - 1; y > x; y--) {
12          distances[y] = distances[y - 1];
13          labels[y] = labels[y - 1];
14        }
15
16        distances[x] = d;
17        labels[x] = g.label;
18
19        break;
20      }
21    }
22  }
23 }
```

```
24
25 // ...
```

Trecho de código 1: Cálculo de distância e ordenação.

Dessa forma, para cada novo valor encontrado com valor de distância inferior a uma distância já armazenada, ou ainda não armazenada de  $k$ , o array de distâncias de tamanho  $k$  é shiftado para a direita para incluir o novo valor de distância, e não perder informação.

Resultar um *array* de distâncias ordenados ao final da aplicação do algoritmo de KNN parece ser uma boa solução, porém ao aplicar paralelismo para calcular a distância entre pontos, não é possível fazer a ordenação do *array* de distâncias e *labels* de tamanho  $k$ , pois esse possui dependência de elementos *passado*, como mostra o Trecho do Código 2.

```
1 distances[y] = distances[y - 1];
2 labels[y] = labels[y - 1];
```

Trecho de código 2: Dependência da distância anterior.

Uma **solução**, aplicando a metodologia PCAM, foi criado um array de *distances* dentro da *struct Group*, e inicializada com valor de  $-1$ , assim, podemos realizar o cálculo em paralelo de cada *group*, esse é o agrupamento, e armazenar cada distância entre cada ponto do grupo em relação à  $P$  nesse *array* de *distances*, como mostra o Trecho de Código 3.

```
1 typedef struct {
2     char label;
3     int length;
4     Point * points;
5     float * distances;
6 } Group;
7
8 //...
9
10 Group parse_next_group() {
11     char label;
12     int length;
13
14     if (scanf(" label=%c ", &label) != 1) on_error();
15     if (scanf(" length=%d ", &length) != 1) on_error();
16
17     Group group;
18     group.label = label;
19     group.length = length;
20     group.points = (Point *) malloc(sizeof(Point) * length);
21     group.distances = (float *) malloc(sizeof(float) * length);
22
23     for (int i = 0; i < length; i++) {
24         group.points[i] = parse_point();
25         group.distances[i] = -1;
26     }
27
28     return group;
29 }
```

Trecho de código 3: Alteração do *Group*.

Dessa forma, deve-se ser enviado ao *thread master* somente o *group*, e essa, por sua vez paralelizará o cálculo das distâncias, como mostra o Trecho de Código 4. Os *workers* por sua vez retornarão o grupo com a distâncias preenchidas. Então será necessário concatenar o resultado de todas as distâncias de todos os grupos, armazenando também a informação da *label* que pertence, para isso pode-se criar uma *struct* para armazenar essas informações em conjunto, como mostra o Trecho de Código 5

```
1 #pragma omp parallel for schedule(dynamic) num_threads(12)
2 for (int i = 0; i < n_groups; i++) {
3     Group g = groups[i];
4
5     for (int j = 0; j < g.length; j++) {
6         float d = euclidean_distance_no_sqrt(to_evaluate, g.points[j]);
7         g.distances[j] = d;
8     }
9 }
```

Trecho de código 4: Armazenamento da distância após alteração do *Group*.

```
1 typedef struct {
2     float distance;
3     char label;
4 } GroupDistLabel;
```

Trecho de código 5: Estrutura de dados para carregar informação de *distance* e *label*.

Como comentado anteriormente, esse trecho do código, em resumo, faz a contagem da quantidade de pontos (tamanho de cada grupo) e aloca em memória um *array* para armazenar *distância* e *label* de cada ponto com esse tamanho. Esse trecho, não inicial, é uma parte sequencial do problema, e portanto, deve ser discriminado da parte paralelizável ao realizar a avaliação de desempenho.

```

1  int total_lenght = 0;
2  for (int i = 0; i < n_groups; i++){
3      total_lenght += groups[i].length;
4  }
5
6  GroupDistLabel* all_groups_dist_label = malloc(sizeof(GroupDistLabel) * total_lenght);
7  int x = 0;
8
9  for (int i = 0; i < n_groups; i++) {
10     Group g = groups[i];
11     for (int j = 0; j < g.length; j++) {
12         all_groups_dist_label[x].distance = g.distances[j];
13         all_groups_dist_label[x].label = g.label;
14         x++;
15     }
16 }

```

Trecho de código 6: Parte sequencial.

As últimas etapas são a utilização do *quicksort*, que foi disponibilizado pelo moodle, que por sua vez deve ser adaptado para receber um *array* de *GroupDistLabel*, bem como ordenar e comparar utilizando *.distance*, sem perder a informação do *label*, e que por sua vez pode ser paralelizável, utilizando OpenMP.

```

1  #pragma omp parallel num_threads(16)
2  {
3      #pragma omp single
4      {
5          quicksort_gdl(all_groups_dist_label, 0, (total_lenght - 1));
6      }
7  }

```

O *quicksort\_gdl(...)* nada mais é que o *quicksorter* compartilhado pelo moodle adaptado para receber o *array GroupDistLabel*.

Por fim, após o sort, basta extrair os primeiros *k* elementos do *array all\_groups\_distances[x].label* em um *array* de *labels* que será contabilizado, e testado, pelo Trecho de Código 7, que foi fornecido pelo problema, além do mais esse trecho de código também é sequencial e, para avaliação do desempenho, deve ser discriminado como parte serial e sequencial do código.

```

1  char labels[k];
2  for (int i = 0; i < k; i++) {
3      labels[i] = all_groups_dist_label[i].label;
4  }
5
6  char most_frequent = labels[0];
7  int most_frequent_count = 1;
8  int current_frequency = 1;
9
10 for (i = 1; i < k; i++) {
11     if (labels[i] != labels[i - 1]) {
12         if (current_frequency > most_frequent_count) {
13             most_frequent = labels[i - 1];
14             most_frequent_count = current_frequency;
15         }
16
17         current_frequency = 1;
18     } else {
19         current_frequency++;
20     }
21
22     if (i == k - 1 && current_frequency > most_frequent_count) {
23         most_frequent = labels[i - 1];
24         most_frequent_count = current_frequency;
25     }
26 }

```

Trecho de código 7: Teste de maior frequência.

Ainda, utilizando o dataset sugerido *python3 main.py 10000 100 400 60000*, obteve-se as seguintes curvas de speed-up, representadas pelas Figuras 2 e 3.

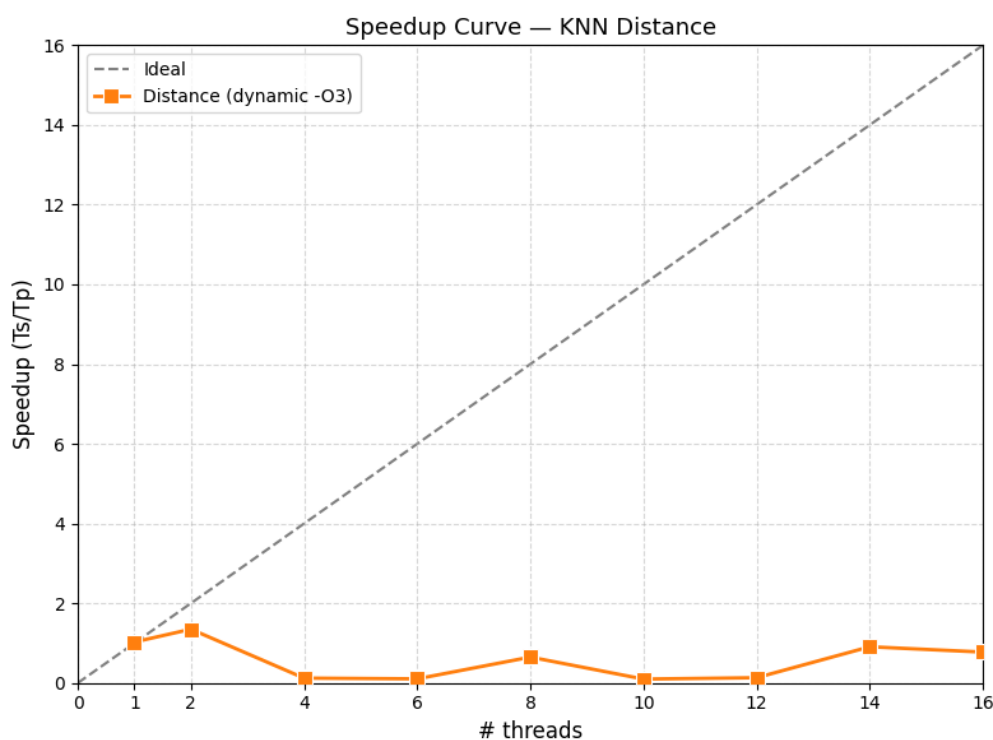


Figura 2: KNN Distance Parallel

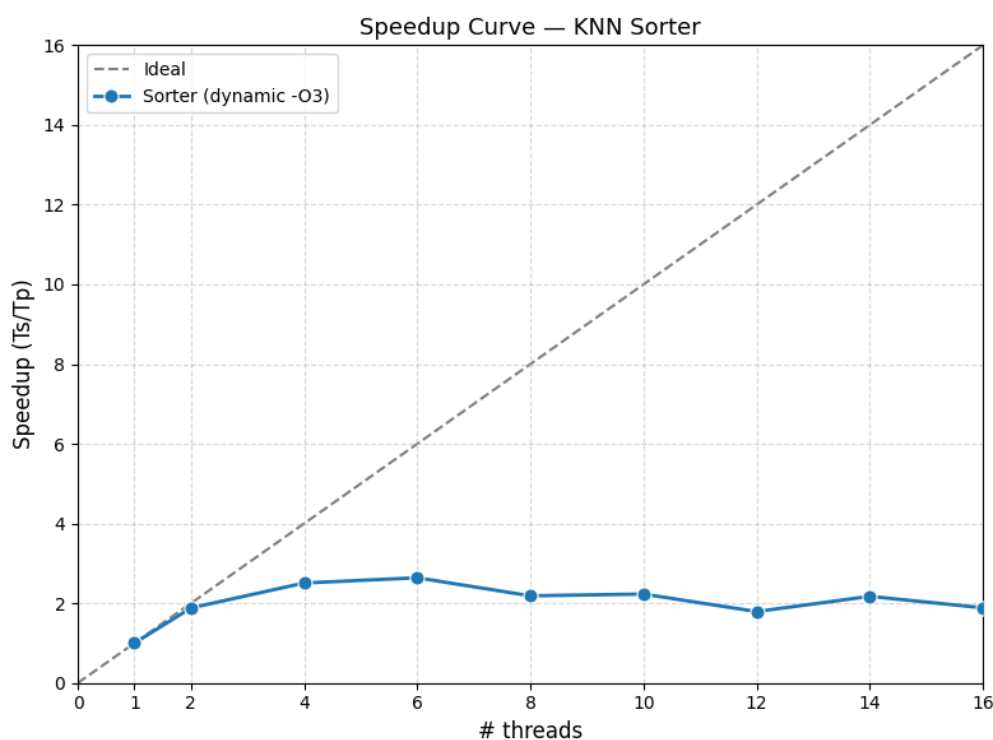


Figura 3: KNN Sorter Parallel

O comportamento da curva do KNN Distance é de fato intrigante, com o aumento da quantidade de *threads* houve uma piora em relação ao tempo serial, ou seja, a curva oscilou em valores abaixo de 1 de *speed-up*, ou seja, a execução das tarefas de cálculo da distância foram iguais ou minimamente piores do que realizar o cálculo de maneira em série. Uma hipótese para esse comportamento, é que a granularidade do processamento em diversas *threads*, para o cálculo da distância euclidiana, que por sua vez acessam variáveis em regiões diferentes da memória, uma vez que o grupo não foi criado alinhado em memória, há uma maior troca de informação com a cache. Além do mais, o cálculo da distância euclidiana é simples e muito rápido, então a

troca de contexto entre *threads* acaba tendo mais influência no tempo de processamento do que no próprio cálculo.

Já o comportamento da curva KNN Sorter mostrou um aumento significativo no *speed-up*, porém esse não acompanhou a curva de *speed-up* ideal, uma hipótese, como comentado no relatório anterior, é de que o sistema operacional, e/ou hardware está capando a execução, poupando recursos, mas ainda pode-se mensurar um aumento do *speed-up* com o aumento do número de *threads* até certo ponto, em que há uma diminuição do *speed-up*. Esse aumento se dá pelo benefício do paralelismo, que dilui o trabalho da movimentação de dados pelos *swaps*, sub dividindo o array *target* em seus pivôs, em múltiplas *threads*.

O código utilizado está disponível no repositório *github* com acesso em:

[https://github.com/affonsoviccari/udesc\\_master\\_ppgca\\_course\\_ppa/tree/master/4\\_repos/c\\_cpp/6\\_knn/algorithm](https://github.com/affonsoviccari/udesc_master_ppgca_course_ppa/tree/master/4_repos/c_cpp/6_knn/algorithm)

Para executar o código, é possível utilizar *make*:

```
1 make clean
2 make
3 ./knn < load.in
```